

Simulateur de stratégie CDF de robotique

Guide du développeur d'agents

Contents

1	Présentation	2
2	Le jeu, dans les grandes lignes	2
3	Règles du jeu en détail	2
4	Premiers pas	3
4.1	Lancer une partie	3
4.2	Choisir quel agent joue	3
5	Écrire son propre agent	4
5.1	Mémoire de l'agent	4
6	L'interface de l'agent (module interface)	5
6.1	Fonctions d'information	5
6.2	Fonctions d'action	5
7	Classes de données	6
8	Référence des constantes (module config)	6

1 Présentation

Un jeu à 2 joueurs simulant la Coupe de France de Robotique 2026, destiné à être joué par des agents Python. Deux robots circulaires s'affrontent sur une carte rectangulaire pour collecter, recolorer et déposer des « caisses » afin de marquer des points.

En tant que développeur d'agent, vous allez :

- écrire des modules Python dans `agents/` qui décident de l'action de votre robot à chaque tick,
- choisir quel agent joue quel joueur dans `agents_config.py`,
- lancer et observer des parties grâce au lanceur `play.py`.

2 Le jeu, dans les grandes lignes

La carte

Un rectangle contenant des zones rectangulaires sans chevauchement : la zone du joueur 0, la zone du joueur 1, et une ou plusieurs *zones de score*.

Caisses

Des rectangles de taille identique dispersés sur la carte. Chaque caisse a une **couleur** (0 ou 1) et peut être ramassée par les robots.

Robots

Chaque joueur contrôle un robot circulaire qui démarre dans sa propre zone. Un robot peut tourner librement, avancer, ramasser une caisse proche, changer la couleur d'une caisse proche, ou poser une caisse qu'il transporte. Avancer, ramasser, recolorer et poser prennent du temps réel : ces actions placent le robot en période de *cooldown* pendant laquelle il ne peut plus agir du tout (voir la Section 3).

Tours

Le temps est discrétisé en *ticks*. À chaque tick, le joueur 0 puis le joueur 1 appelle `make_decision()` une fois. **Exactement une action** peut être effectuée par tour.

Score

Calculé à la fin de la partie :

- chaque caisse posée dans la zone du joueur X fait gagner `+POINTS_OWN_AREA` points à X (quelle que soit la couleur de la caisse) ;
- chaque caisse posée dans une zone de score et dont la couleur est X fait gagner `+POINTS_SCORING_AREA` points à X ;
- les caisses actuellement transportées par un robot ne rapportent aucun point.

3 Règles du jeu en détail

- **Une action par tour** : `make_decision()` peut appeler exactement une fonction d'action. `rotate` ne compte pas comme une action et peut être appelée un nombre arbitraire de fois à chaque tour (sous réserve de la règle de coût temporel ci-dessous).
- **Emplacement de pose** : à une distance fixe (`LAY_DOWN_DISTANCE`) directement devant le robot, orientée selon la direction actuelle du robot. Échoue si l'emplacement est hors de la carte ou chevauche une autre caisse.
- **Rotation des caisses** : les caisses prennent l'orientation du robot lorsqu'elles sont posées.

- **Disposition initiale** : les positions, orientations et zones sont fixes et entièrement définies dans le fichier de configuration. Les couleurs des caisses sont également définies dans la configuration par défaut, sauf si la partie est lancée avec `python play.py run --random-colors`, qui attribue alors une couleur aléatoire (0 ou 1) à chaque caisse.
- **Les actions échouées ne consomment pas le tour** : si une action lève `GameError` (par ex. `lay_down` bloqué, `pickup` hors de portée), votre tour n'est *pas* consommé, vous pouvez donc essayer autre chose.
- **Coût temporel** : `move`, `pickup`, `set_color` et `lay_down` occupent chacune le robot pendant un nombre de ticks donné par leur propre constante — `MOVE_COOLDOWN`, `PICKUP_COOLDOWN`, `SET_COLOR_COOLDOWN` et `LAY_DOWN_COOLDOWN` respectivement. Une valeur N signifie que le robot est occupé pendant N ticks *au total*, y compris le tick où l'action réussit (donc $N = 1$ ne donne aucun délai supplémentaire, et $N > 1$ donne $N - 1$ ticks supplémentaires de gel). Tant que le robot est occupé, **toute** fonction d'action — y compris `rotate` — lève `GameError` sans consommer le tour. Appelez `get_cooldown(player)` pour connaître le nombre de ticks restants (0 = libre).
- **Collision lors du déplacement** : le robot s'arrête juste avant de heurter le bord de la carte, l'autre robot, ou une caisse posée.
- **Les caisses portées par un robot ne rapportent aucun point** et ne sont pas des obstacles physiques pour le déplacement.

4 Premiers pas

4.1 Lancer une partie

Utilisez `play.py` avec votre interpréteur Python habituel (`python` / `python3` / `py`) depuis la racine du projet.

```
python play.py run                # lance une partie, affiche la
    progression et le score final
python play.py run --quiet        # supprime les messages d'erreur
    a chaque tick
python play.py run --save         # sauvegarde aussi la partie
    dans saved_games/
python play.py run --save --view  # lance, sauvegarde, puis rejoue
    immédiatement la partie
python play.py run --random-colors # tire au hasard la couleur
    initiale de chaque caisse
python play.py view <history.json> # rejoue une partie depuis
    saved_games/ (ou un chemin quelconque)
```

4.2 Choisir quel agent joue

Modifiez `agents_config.py` à la racine du projet :

```
PLAYER0_AGENT = "agents.simple_agent"
PLAYER1_AGENT = "agents.idle_agent"
```

Chaque constante pointe vers un module du dossier `agents/`, chargé dynamiquement. Les deux emplacements peuvent désigner le *même* module pour faire jouer un agent contre lui-même.

5 Écrire son propre agent

Créez `agents/my_agent.py` en y définissant une classe `Agent` dotée d'une méthode `make_decision(self)`. N'oubliez pas d'importer `interface` et `config`.

```
import interface
import config

class Agent:
    def make_decision(self):
        player = interface.me()
        accessible = interface.get_accessible_boxes(player)
        if accessible:
            interface.pickup(accessible[0].id)
        else:
            interface.move(10.0)
```

Pointez ensuite `agents_config.PLAYER0_AGENT` (ou `PLAYER1_AGENT`) vers `"agents.my_agent"` et lancez `python play.py run --view`.

Consultez `agents/random_agent.py` pour un exemple.

5.1 Mémoire de l'agent

Le lanceur crée une *instance* `Agent()` par joueur, même lorsque `PLAYER0_AGENT` et `PLAYER1_AGENT` désignent le même module. Votre agent peut donc avoir une mémoire si vous en avez besoin :

```
class Agent:
    def __init__(self):
        self.visited = set()

    def make_decision(self):
        player = interface.me()
        self.visited.add(interface.get_position(player))
        # ...
```

6 L'interface de l'agent (module `interface`)

6.1 Fonctions d'information

Peuvent être appelées librement, autant de fois que voulu.

Fonction	Retourne
<code>me()</code>	L'indice du joueur de cet agent (0 ou 1)
<code>opponent()</code>	<code>1 - me()</code>
<code>get_position(player)</code>	Centre (<code>x</code> , <code>y</code>) du robot du joueur
<code>get_orientation(player)</code>	Vecteur unitaire (<code>x</code> , <code>y</code>) indiquant la direction du robot du joueur
<code>get_map()</code>	L'objet <code>Map</code> (<code>.width</code> , <code>.height</code> , <code>.areas</code>)
<code>get_area_containing(position)</code>	La zone <code>Area</code> contenant <code>position</code> , ou <code>None</code> si elle n'est dans aucune zone
<code>get_area(area_id)</code>	La zone <code>Area</code> ayant cet identifiant, ou <code>None</code>
<code>get_area_center(area_id)</code>	Centre (<code>x</code> , <code>y</code>) de cette zone
<code>get_boxes()</code>	Tous les objets <code>Box</code> (au sol et transportés)
<code>get_box(box_id)</code>	La caisse <code>Box</code> ayant cet identifiant, ou <code>None</code>
<code>get_accessible_boxes(player)</code>	Caisses au sol à portée de <code>player</code>
<code>get_boxes_held(player)</code>	Caisses actuellement transportées par <code>player</code>
<code>is_accessible(player, box_id)</code>	La caisse est-elle au sol et à portée ?
<code>is_colliding(player, amount)</code>	Avancer de <code>amount</code> provoquerait-il une collision (bord/robot/caisse) ?
<code>get_obstacle_distance(player)</code>	Distance entre le robot de <code>player</code> et l'obstacle le plus proche (bord de la carte, autre robot, ou caisse posée) dans la direction actuelle ; non limitée par <code>MAX_MOVE_SPEED</code> , peut valoir <code>inf</code>
<code>get_box_distance(player, box_id)</code>	Distance cercle-rectangle, du robot vers la caisse
<code>get_box_direction(player, box_id)</code>	Vecteur unitaire du centre du robot vers le centre de la caisse
<code>get_area_distance(player, area_id)</code>	Distance du centre du robot au rectangle de la zone (0 si à l'intérieur)
<code>get_area_direction(player, area_id)</code>	Vecteur unitaire du centre du robot vers le centre de la zone
<code>get_score(player)</code>	Score que <code>player</code> obtiendrait si la partie se terminait maintenant
<code>get_cooldown(player)</code>	Nombre de ticks restants avant que le robot de <code>player</code> puisse agir à nouveau (0 = libre maintenant)

6.2 Fonctions d'action

Exactement une par appel à `make_decision()` (sauf `rotate`, qui ne compte pas comme une action). Les cinq, y compris `rotate`, sont bloquées tant que le robot est en cooldown

Fonction	Effet
<code>move(amount)</code>	Avance de <code>amount</code> (≥ 0 ; plafonné à <code>MAX_MOVE_SPEED</code>) ; s'arrête juste avant toute collision. Déclenche un cooldown de <code>MOVE_COOLDOWN</code> ticks
<code>rotate(new_direction)</code>	Orienté le robot vers <code>new_direction</code> . Ne déclenche elle-même aucun cooldown
<code>pickup(box_id)</code>	Ramasse une caisse accessible (échoue si trop loin ou si les mains sont pleines). Déclenche un cooldown de <code>PICKUP_COOLDOWN</code> ticks
<code>set_color(box_id, color)</code>	Recolore une caisse accessible en 0 ou 1. Déclenche un cooldown de <code>SET_COLOR_COOLDOWN</code> ticks
<code>lay_down(box_id)</code>	Pose une caisse transportée devant le robot (échoue si bloqué / hors limites). Déclenche un cooldown de <code>LAY_DOWN_COOLDOWN</code> ticks

Toutes les fonctions d'action peuvent lever :

- `interface.ActionError` si une seconde action est tentée durant le même tour.
- `game.GameError` si l'action elle-même est invalide (hors de portée, bloquée, valeur de `move` négative, etc.) **ou si le robot est encore en cooldown suite à une action précédente** (cela s'applique aussi à `rotate`). Cela ne consomme **pas** le tour, vous pouvez donc vous rattraper et essayer autre chose.

7 Classes de données

Ce sont les objets retournés par les fonctions de `interface`.

Box(id, position, orientation, color, owner)

Une caisse sur la carte. `owner == -1` signifie qu'elle est posée au sol ; sinon `owner` est l'indice du joueur qui la transporte actuellement. `color` vaut 0 ou 1 ; `orientation` est un vecteur unitaire le long de son axe de largeur.

Area(id, type, x, y, width, height)

Une zone rectangulaire alignée sur les axes. `type` vaut 0 (zone du joueur 0), 1 (zone du joueur 1), ou 2 (zone de score).

Map(width, height, areas)

La carte statique : dimensions globales plus la liste de tous les objets `Area` (zones des joueurs et zones de score confondues).

8 Référence des constantes (module `config`)

Carte

Constante	Valeur	Signification
<code>MAP_WIDTH</code>	1000.0	Largeur du terrain de jeu, en unités de jeu (étendue selon <code>x</code> ; pour rappel, (0, 0) est le coin supérieur gauche).

Constante	Valeur	Signification
MAP_HEIGHT	600.0	Hauteur du terrain de jeu (étendue selon y).

Zones

Constante	Valeur	Signification
PLAYER0_AREA	(0, 200, 150, 200)	Rectangle (x, y, largeur, hauteur) définissant la zone de base du joueur 0.
PLAYER1_AREA	(850, 200, 150, 200)	Idem, pour le joueur 1 (<code>Area.type == 1</code>).
SCORING_AREAS	liste de 2 rectangles	Liste de rectangles (x, y, largeur, hauteur), chacun devenant une <code>Area</code> avec <code>type == 2</code> .

Positions et orientations de départ des robots

Constante	Valeur	Signification
PLAYER0_START	(75, 300)	Position de départ (x, y) du robot du joueur 0 — à l'intérieur de <code>PLAYER0_AREA</code> .
PLAYER0_START_ANGLE	0.0	Angle d'orientation initial du robot du joueur 0, en degrés mesurés depuis l'axe +x (0° = orienté vers la droite, vers le centre de la carte).
PLAYER1_START	(925, 300)	Position de départ du robot du joueur 1.
PLAYER1_START_ANGLE	180.0	Angle d'orientation initial du robot du joueur 1 (180° = orienté vers la gauche, vers le centre de la carte).

Physique des robots

Constante	Valeur	Signification
ROBOT_RADIUS	20.0	Rayon du corps circulaire du robot.
MAX_MOVE_SPEED	10.0	La plus grande valeur de <code> amount </code> qu'un appel à <code>move</code> appliquera réellement, par tick.
MAX_BOXES_HELD	3	Nombre maximal de caisses qu'un robot peut transporter à la fois.

Dimensions des caisses

Constante	Valeur	Signification
BOX_WIDTH	40.0	Largeur d'une caisse.
BOX_HEIGHT	25.0	Hauteur d'une caisse.

Distances d'interaction

Constante	Valeur	Signification
ACCESSIBILITY_RADIUS	25.0	Distance cercle-rectangle maximale (du centre du robot au point le plus proche de la caisse) en deçà de laquelle une caisse est considérée comme « accessible » (c.-à-d. à portée pour <code>pickup</code> et <code>set_color</code>).
LAY_DOWN_DISTANCE	45.0	Distance fixe entre le centre du robot et le <i>centre</i> d'une caisse posée via <code>lay_down</code> .

Score

Constante	Valeur	Signification
POINTS_OWN_AREA	3	Points attribués au joueur <i>X</i> pour chaque caisse posée dans la zone de <i>X</i> .
POINTS_SCORING_AREA	5	Points attribués au joueur <i>X</i> pour chaque caisse posée dans une zone de score dont la couleur correspond à <i>X</i> .

Timing

Constante	Valeur	Signification
DT	0.1	Nombre de secondes représentées par un tick.
GAME_DURATION	60.0	Durée totale de la partie, en secondes (= DT × TOTAL_TICKS).
TOTAL_TICKS	600	Nombre de ticks dans une partie.

Coûts d'action (temporels)

Constante	Valeur	Signification
MOVE_COOLDOWN	1	Nombre total de ticks pendant lesquels <code>move</code> occupe le robot, y compris le tick où elle réussit.
PICKUP_COOLDOWN	20	Nombre total de ticks pendant lesquels <code>pickup</code> occupe le robot, y compris le tick où elle réussit.
SET_COLOR_COOLDOWN	10	Nombre total de ticks pendant lesquels <code>set_color</code> occupe le robot, y compris le tick où elle réussit.
LAY_DOWN_COOLDOWN	20	Nombre total de ticks pendant lesquels <code>lay_down</code> occupe le robot, y compris le tick où elle réussit.

Tant que l'une de ces temporisations est active (`get_cooldown(player) > 0`), **toutes** les fonctions d'action — y compris `rotate` — lèvent `GameError` sans consommer le tour.

Caisses initiales

Constante	Valeur	Signification
INITIAL_BOXES	liste de 12 tuples	La disposition de départ fixe : chaque entrée est <code>(x, y, angle_degrees, color)</code> — la position initiale du centre de la caisse, son orientation exprimée comme un angle en degrés depuis l'axe $+x$, et sa couleur de départ (0 ou 1).